

Adversarial Attacks and Defenses on Graph-aware Large Language Models (LLMs)

Anonymous submission

Abstract

Large Language Models (LLMs) are increasingly coupled with graph-structured inputs for tasks such as node classification, a setting historically dominated by Graph Neural Networks (GNNs). Although this integration can exploit relational information to improve predictive performance, the adversarial robustness of graph-aware LLMs has not been systematically characterized. We provide an initial technical study of this vulnerability surface by applying existing graph adversarial attack procedures, including poisoning attacks at training time and evasion attacks at test time, to two representative graph-aware LLMs, LLAGA (Chen et al. 2024) and GRAPHPROMPTER (Liu et al. 2024). We further identify an additional attack vector specific to LLAGA, in which an adversary injects malicious nodes into placeholder positions of the node sequence template and thereby substantially degrades model accuracy. Our analysis shows that graph-encoding design choices directly modulate attack effectiveness: (1) the node sequence template in LLAGA amplifies vulnerability; (2) the GNN encoder used in GRAPHPROMPTER provides comparatively stronger structural robustness; and (3) both architectures remain vulnerable to imperceptible textual feature perturbations. Finally, we introduce GALGUARD, an end-to-end defense framework that combines an LLM-based feature-correction module for feature-level perturbations with adapted GNN defenses for structural attacks¹

1 Introduction

Graphs encode entities as nodes and relationships as edges, and therefore provide a formal representation for complex relational systems in domains such as social networks (Fan et al. 2019), biology (Dong, Mücke, and Khosla 2022), finance (Wang et al. 2021), and healthcare (Ahmedt-Aristizabal et al. 2021). Graph Neural Networks (GNNs) (Hamilton, Ying, and Leskovec 2017; Kipf and Welling 2022; Veličković et al. 2018) are architectures specialized for such graph-structured inputs and have become standard models for tasks including node classification and link prediction. More recently, large language models (LLMs), including GPT-4 (Achiam et al. 2023), Gemini (Team et al. 2023), LLaMA (Touvron et al. 2023; Dubey et al. 2024), Vicuna (Chiang et al. 2023), and PaLM-2 (Anil et al. 2023), have motivated the integration of graph data

with pretrained textual representations. This integration, commonly referred to as *graph-aware LLMs* (Xie et al. 2023), defines a research direction in which LLMs are adapted to execute tasks that were previously addressed primarily by GNNs (Ye et al. 2023; Fatemi, Halcrow, and Perozzi 2023; Kuang et al. 2023; Zhang et al. 2023; Chen et al. 2023b, 2024). Empirical studies further report that graph-aware LLMs can exceed GNN performance on multiple graph-related benchmarks (Wenkel, Wolf, and Knyazev 2023; Chen et al. 2023b; Zhu et al. 2024; Chen et al. 2024), making them technically plausible alternatives to conventional graph models.

Despite this increasing adoption, the attack surface of graph-aware LLMs remains insufficiently specified. Existing adversarial attacks on GNNs, including poisoning attacks applied during training and evasion attacks applied during inference (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Dai et al. 2023), are commonly formulated in a black-box regime in which the adversary lacks access to model internals. These attacks compromise *model integrity* by inducing performance degradation through small modifications to graph structure or node features. Whereas prior GNN robustness work has emphasized structural perturbations, the textual feature channel used by graph-aware LLMs introduces an additional vulnerability class: imperceptible changes to node text that preserve human readability while altering the token sequence processed by the model.

These observations motivate our central research question: *whether adversarial attacks designed for GNNs remain effective against the emerging graph-aware LLM paradigm*. Answering this question requires explicit consideration of the graph-to-LLM interface. Node features can be supplied to an LLM through its native textual modality, but graph topology must be encoded through an additional mechanism that preserves relational information (Chen et al. 2024; Guo et al. 2023; Liu and Wu 2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024). We therefore first partition existing graph-aware LLM methods into two encoding classes: methods that serialize graph information as *textual descriptions*, and methods that use *learned projectors* to map graph representations into the LLM input space. Because standard text-based attacks are already applicable to models that receive purely textual graph descriptions, our empirical study focuses on learned-projector methods. We

¹Our code will be released upon acceptance.

instantiate this class with two computationally efficient representative systems, LLAGA (Chen et al. 2024) and GRAPH-PROMPTER (Liu et al. 2024), both of which use frozen pre-trained LLM backbones.

Our results show that specific graph-encoding decisions can increase adversarial susceptibility rather than merely determine representational capacity. In particular, we identify a previously uncharacterized attack surface in LLAGA: malicious nodes can be inserted into placeholder positions of the node sequence template, thereby corrupting the representation consumed by the LLM and substantially reducing accuracy. We also propose GALGUARD, an end-to-end graph-aware LLM defense that composes an LLM-based feature corrector with adapted GNN-based structural defenses. To our knowledge, this is the first systematic investigation of adversarial vulnerability in graph-aware LLMs. The contributions of the paper are summarized as follows:

- We apply representative GNN adversarial attacks, specifically NETTACK (Zügner, Akbarnejad, and Günnemann 2018) and METAATTACK (Zügner and Günnemann 2019), to quantify the transferability of poisoning and evasion attacks to graph-aware LLMs.
- We define a graph-encoding taxonomy for graph-aware LLMs that distinguishes how different methods integrate graph structure into LLM input representations.
- We identify a new attack surface in LLAGA, showing that malicious nodes inserted as placeholders in the node sequence template can degrade predictive performance.
- We provide a systematic vulnerability assessment of graph-aware LLMs under imperceptible perturbations to textual node features, an attack channel typically omitted in GNN-centered analyses.
- We propose GALGUARD, an end-to-end defense strategy that integrates LLM-based feature correction with adapted GNN-based structural defenses.

2 Graph Encoding-Based Taxonomy of LLM Adaptations

Graph encoding methods for LLMs can be categorized into two primary mechanisms: *textual descriptions of graphs*, in which graph information is encoded as natural language, and *learned graph projectors*, in which a trainable projector maps graph data into embeddings or structured token-compatible representations. We summarize the representative encoding mechanisms in each category, with Figure 5 illustrating the distinction. A more detailed taxonomy appears in Appendix A.1, and related work on GNN attacks appears in Appendix A.2.

Graph Representation as a Textual Description. Textual-description methods serialize graph data into natural language so that the LLM can process the graph through its standard prompt interface, typically under task-specific instructions (Guo et al. 2023; Liu and Wu 2023; Ye et al. 2023; Wenkel, Wolf, and Knyazev 2023; Wei et al. 2024b; Fatemi, Halcrow, and Perozzi 2023; Tan et al. 2023; Zhao et al. 2023; Wei et al. 2024a; Zhang 2023). In multimodal settings, Liu et al. (2023) and Su et al. (2022) associated molecular graphs

with textual descriptions. In addition, Huang et al. (2023a) studied the insertion of graph structure into LLM prompts and found that LLMs tend to interpret graph-related prompts as contextual paragraphs, which makes prompt design a central component of the modeling pipeline (Brannon et al. 2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024; He et al. 2024). Other work uses LLMs to refine node features, augment graph structures, function as predictors, or provide feature extraction for GNNs (Chen et al. 2023a; Chandra et al. 2020; Chen et al. 2023b; Mavromatis et al. 2023). For inference, Zhu et al. (2024) proposed a fine-tuning method for textual graphs that combines LLMs with GNNs, and Duan et al. (2023) introduced an approach that improves textual graph learning. Although these methods provide a direct interface between LLMs and graph tasks, they can fail to encode fine-grained structural information, particularly on large graphs, and their performance is strongly coupled to prompt engineering.

Graph Representation as a Learned Projector. Learned-projector methods instead map graph-derived representations into embeddings or graph vectors that can be consumed by the LLM input interface. This category includes LLAGA, which constructs node-level templates and applies a linear projector (Chen et al. 2024), as well as methods that insert GNN layers into a pre-trained LLM pipeline (Liu et al. 2024; Qin et al. 2023). Related approaches encode knowledge graphs through graph neural prompts (Tian et al. 2024), use prompt-based node feature extractors with a GNN adapter (Huang et al. 2023b), or compress graph information into fixed-length prefixes using graph transformers (Chai et al. 2023; Yang et al. 2021; Peng et al. 2024).

This Work: We restrict the empirical scope to projector-based methods with *frozen pre-trained LLMs*, because this design reduces training cost while preserving adaptability across graph tasks (Chen et al. 2024; Liu et al. 2024; Qin et al. 2023; Tian et al. 2024; Huang et al. 2023b). We evaluate two representative instances: LLAGA (Chen et al. 2024), which combines node-level templates with a linear projector, and GRAPH-PROMPTER (Liu et al. 2024), which combines a GNN encoder with a linear projector. We exclude textual-description methods because their scalability limitations and prompt dependence do not expose the projector-specific graph vulnerabilities studied in this work.

3 Threat Model

We define the threat model by extending established graph adversarial formulations to graph-aware LLMs (Zügner, Akbarnejad, and Günnemann 2018; Wu et al. 2019; Zügner and Günnemann 2019; Ma, Ding, and Mei 2020; Jin et al. 2021; Dai et al. 2023; Li et al. 2023). In a social-network setting, for example, an adversary may control only a small number of bot accounts; this constraint corresponds to manipulating a limited subset of nodes in order to evade detection or alter the classification of legitimate users. In a Wikipedia-like graph, sparsely connected hoax articles may be linked strategically to legitimate articles so that the predicted categories of genuine content are changed. Such transferability is consequential because graph-aware LLMs may be deployed in recommendation systems, social-media analysis, healthcare, and

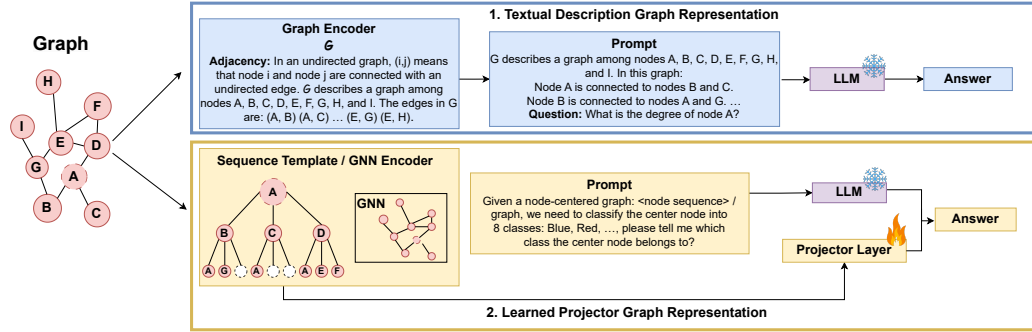


Figure 1: **Graph encoding-based adaptations of LLMs.** In the textual-description representation, graph information is serialized as text (top). In the learned-projector representation, graph information is encoded either through node templates that convert neighborhoods into sequences or through GNNs (bottom). In both cases, the LLM remains frozen and training is restricted to the projector that maps graph representations into tokens.

other settings where they could replace GNN components. Following prior graph-attack work, we therefore specify the threat model along four components: model access, adversarial capabilities, attack strategies, and adversarial objective.

Model Access. The attacker is assumed to operate in a black-box regime and has no access to the target architecture, learned parameters, or internal representations. The attacker can affect the model only through modifications to training data or inference-time inputs.

Adversarial Capabilities. Consistent with standard poisoning and evasion formulations, the attacker has access to the target model’s training data in the poisoning case and inference data in the evasion case. Subject to a fixed perturbation budget and unnoticeability constraints, the attacker may modify existing data or inject additional data.

Attack Strategies. The attacker uses two perturbation classes: feature manipulation, which alters node attributes, and structural manipulation, which adds or removes graph edges.

Attacker’s Goal. The attacker aims to compromise model integrity by reducing overall classification accuracy, equivalently increasing the misclassification rate of the target classifier.

4 Experimental Setup

Dataset. We evaluate on three widely used text-attributed graph benchmarks, Cora, Citeseer, and PubMed (Yang, Cohen, and Salakhudinov 2016). These datasets span small-to-medium graph scales with different sparsity regimes and are standard evaluation instances for the graph-aware LLM methods considered in this study (Chen et al. 2023a, 2024; Liu et al. 2024). To assess behavior at a larger scale, we additionally report an ArXiv experiment (Hu et al. 2020) in Appendix A.5. Following (Chen et al. 2024), we use train/validation/test splits of 6:2:2 for Cora, Citeseer, and PubMed, and 5:2:3 for ArXiv. Dataset statistics are reported in Table 1.

Tasks. We use *node classification* as the primary prediction task, because it is a standard evaluation protocol for graph machine learning systems and the canonical setting for the adversarial graph attacks considered here (Zügner, Akbarne-

Table 1: Structural statistics of the evaluated graph datasets.

Data	#Node	#Edge	Sparsity (%)
Cora	2708	5429	14.81
CiteSeer	3327	4614	8.34
PubMed	19717	44338	2.28

jad, and Günnemann 2018; Zügner and Günnemann 2019; Liu and Wu 2023).

Evaluation Metrics. We report *accuracy*, defined as the proportion of nodes whose predicted labels match the ground-truth labels. Each experiment is repeated three times, and all tables report the mean and standard deviation across runs. All experiments were executed on a machine equipped with an NVIDIA A40 GPU with 48GB of memory.

Choice of Graph-aware LLM and Adversarial Attack Methods. We specify the model and attack selections used to evaluate adversarial vulnerability in graph-aware LLMs.

Graph-Aware LLM Methods. We evaluate two representative graph-aware LLM methods, LLAGA (Chen et al. 2024) and GRAPH PROMPTER (Liu et al. 2024). Both systems keep the pre-trained LLM frozen, which reduces optimization cost and permits reuse across graph tasks and LLM backbones. LLAGA constructs node-level templates and maps them through a linear projector, whereas GRAPH PROMPTER uses a GNN-based encoder followed by a linear projector. In both implementations, the projector is a 2-layer multi-layer perceptron (MLP). The foundation LLMs used by LLAGA and GRAPH PROMPTER are Vicuna-7B (Chiang et al. 2023) and LLAMA2-7B (Touvron et al. 2023), respectively.

Adversarial Attack Methods. We instantiate poisoning and evasion attacks using two representative graph adversarial methods, NETTACK (Zügner, Akbarnejad, and Günnemann 2018) and METAATTACK (Zügner and Günnemann 2019). NETTACK is a targeted procedure that searches for minimal perturbations causing selected nodes to be misclassified, whereas METAATTACK is an untargeted procedure that optimizes the graph structure as a hyperparameter. For NET-

Table 2: Accuracy of LLAGA and GRAPH PROMPTER under NETTACK and METAATTACK on Cora, Citeseer, and PubMed.

Attack	Type	Cora		Citeseer		PubMed	
		LLAGA	GRAPH PROMPTER	LLAGA	GRAPH PROMPTER	LLAGA	GRAPH PROMPTER
Clean	—	0.89 $\pm$ 0.07	0.60 $\pm$ 0.03	0.71 $\pm$ 0.04	0.70 $\pm$ 0.03	0.90 $\pm$ 0.04	0.90 $\pm$ 0.02
NETTACK	Poisoning	0.87 $\pm$ 0.04 (2%)	0.60 $\pm$ 0.03 (0%)	0.64 $\pm$ 0.05 (10%)	0.68 $\pm$ 0.02 (3%)	0.89 $\pm$ 0.03 (1%)	0.88 $\pm$ 0.02 (2%)
	Evasion	0.55 $\pm$ 0.09 (38%)	0.53 $\pm$ 0.05 (12%)	0.59 $\pm$ 0.04 (19%)	0.61 $\pm$ 0.03 (14%)	0.84 $\pm$ 0.05 (7%)	0.81 $\pm$ 0.02 (10%)
METAATTACK	Poisoning	0.79 $\pm$ 0.03 (11%)	0.58 $\pm$ 0.05 (3%)	0.63 $\pm$ 0.04 (12%)	0.65 $\pm$ 0.06 (7%)	0.89 $\pm$ 0.08 (1%)	0.85 $\pm$ 0.02 (6%)
	Evasion	0.44 $\pm$ 0.06 (51%)	0.52 $\pm$ 0.04 (13%)	0.50 $\pm$ 0.03 (35%)	0.56 $\pm$ 0.02 (22%)	0.73 $\pm$ 0.04 (19%)	0.77 $\pm$ 0.03 (14%)

TACK, we follow (Li et al. 2023; Zügner and Günnemann 2019) by sequentially targeting each node and aggregating the outcomes, thereby converting the targeted method into an untargeted evaluation protocol. Across experiments, the edge perturbation budget is fixed at 10% to satisfy the unnoticeability constraint used in prior work (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019). Additional procedural details for METAATTACK are provided in Appendix A.2.

5 Results

5.1 Poisoning and Evasion Attacks on Graph-aware LLMs

We evaluate poisoning and evasion attacks by adapting procedures from NETTACK (Zügner and Günnemann 2019) and METAATTACK (Zügner and Günnemann 2019). In the poisoning setting for LLAGA, the perturbed graph is used to generate node-level sequences before retraining the projector. In the poisoning setting for GRAPH PROMPTER, the perturbed graph is re-encoded with a GAT and the projector is retrained on the resulting representations. In the evasion setting, both models are trained on clean data, and perturbations are applied only at inference time.

Results. Table 2 supports three empirical observations. First, poisoning produces limited degradation on graph-aware LLMs, with the largest observed decrease equal to 11% across the evaluated attack methods. This pattern indicates that the learned representations and the contextual information supplied by the LLM can partially absorb training-time perturbations. By contrast, evasion attacks are substantially stronger and reduce accuracy by as much as 51%. Second, METAATTACK generally yields larger degradation than NETTACK in both poisoning and evasion settings, consistent with its optimization of graph edges as hyperparameters. Third, denser graphs exhibit greater vulnerability than sparser graphs: under evasion, LLAGA drops by 51% on the denser Cora dataset but by only 19% on the sparser PubMed dataset.

5.2 Which Graph-aware Paradigm is More Susceptible to Graph Adversarial Attacks and Why?

The results in Table 2 indicate that LLAGA is more susceptible than GRAPH PROMPTER because its input construction is more directly coupled to *structural perturbations*. LLAGA injects graph structure through two mechanisms:

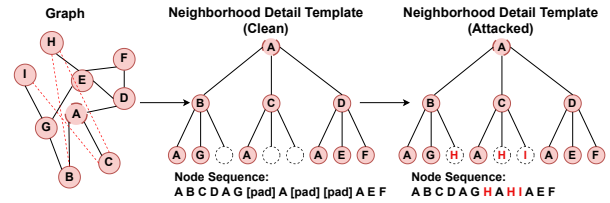


Figure 2: Illustration of node sequence template injection attacks. The displayed instance is the Non-adjacent Injection attack (NI), where red dashed lines denote adversarial edge perturbations. For Supernode Injection (SI) and Multiple Supernode Injection (MSI), the supernode or supernodes are inserted into placeholder positions. Here, $k = 3$.

fixed-length node-level sequences and graph Laplacian embeddings. Each position in a node-level sequence corresponds to a relative structural role in the original graph. Consequently, when the graph is perturbed, the embeddings supplied to the LLM are modified at the representational interface itself, which directly changes the information available for prediction. In GRAPH PROMPTER, by contrast, textual node embeddings remain unchanged under structural perturbations because only the GNN component is affected. The unperturbed textual channel therefore provides auxiliary information that can compensate for perturbations in the structural channel.

6 Node Sequence Template Injection Attacks

We identify a new attack surface in LLAGA by targeting the neighborhood detail template that constructs node sequences. The attack is illustrated in Figure 2, and the corresponding algorithm is provided in Appendix A.4. For a target node u , the neighborhood detail template first constructs a *fixed-shape* computational tree in which each node’s children are sampled 1-hop neighbors. The sampling procedure is repeated recursively for subsequent tree levels. When a node has too few neighbors to fill the required branching factor, placeholder entries are inserted to preserve a fixed size, e.g., $k = 10$ children, matching the original LLAGA design (see “Clean” in Figure 2 and lines 1-14 of the algorithm). The use of fixed placeholder positions creates an exploitable structural interface.

The adversary exploits this template by manipulating the graph so that malicious nodes occupy placeholder positions,

Table 3: Accuracy under node sequence template injection attacks across datasets. *NI*, *SI*, and *MSI* denote non-adjacent injection, single-supernode injection, and multiple-supernode injection, respectively. Clean denotes model performance without attack.

Data	Attack	NI	SI	MSI
Cora	Clean	————	0.89 $\pm$ 0.07	————
	Poisoning	0.86 $\pm$ 0.03	0.84 $\pm$ 0.06	0.82 $\pm$ 0.04
	Evasion	0.42 $\pm$ 0.08	0.36 $\pm$ 0.05	0.30 $\pm$ 0.05
Citeseer	Clean	————	0.71 $\pm$ 0.04	————
	Poisoning	0.69 $\pm$ 0.05	0.66 $\pm$ 0.03	0.62 $\pm$ 0.02
	Evasion	0.36 $\pm$ 0.06	0.30 $\pm$ 0.04	0.24 $\pm$ 0.05
PubMed	Clean	————	0.90 $\pm$ 0.04	————
	Poisoning	0.88 $\pm$ 0.04	0.85 $\pm$ 0.05	0.81 $\pm$ 0.03
	Evasion	0.85 $\pm$ 0.07	0.82 $\pm$ 0.04	0.76 $\pm$ 0.03

thereby degrading the representation used by LLAGA. We define three variants. *Non-Adjacent Injection (NI)* inserts nodes that are outside the 2-hop neighborhood of a target node u (see "Attacked" in Figure 2 and lines 21-27). *Supernode Injection (SI)* inserts a high-degree node as a placeholder while ensuring that it remains outside the target node's 2-hop neighborhood. *Multiple Supernode Injection (MSI)* extends *SI* by inserting multiple distinct supernodes into placeholder positions (lines 35-43). In all cases, uninformative nodes are introduced into the node sequence, corrupting the target-node representation and inducing a poorly trained projection layer, which compromises both training-time learning and inference-time prediction.

Results. As reported in Table 3, LLAGA remains comparatively resilient to poisoning on Cora and Citeseer, but its evasion accuracy decreases sharply. Under *NI*, *SI*, and *MSI*, evasion accuracy falls to 0.42, 0.36, and 0.30 on Cora and to 0.36, 0.30, and 0.24 on Citeseer, respectively.

On the larger PubMed dataset, LLAGA is more robust to injection attacks, with evasion accuracies of 0.85, 0.82, and 0.76. A plausible explanation is that localized injections have higher relative influence in smaller and more tightly connected graphs such as Cora and Citeseer, where they substantially disrupt neighborhood-level reasoning. In the larger PubMed graph, the perturbation has a more diffuse effect and therefore yields smaller accuracy degradation.

Relative to conventional graph adversarial methods, node injection is often more damaging. For example, *MSI* produces evasion accuracies of 0.30, 0.24, and 0.76 on Cora, CiteSeer, and PubMed, respectively, whereas METAATTACK produces 0.44, 0.50, and 0.73 on the same datasets (Table 2).

7 Poisoning and Evasion Attack via Feature Perturbation

Most graph adversarial attacks perturb graph structure (Li et al. 2023; Wu et al. 2019; Dai et al. 2018; Sun et al. 2022; Zügner and Günnemann 2019). Although NETTACK (Zügner, Akbarnejad, and Günnemann 2018) also manipulates features, its feature-space operation does not directly transfer to textual node attributes. Because structural attacks have

only moderate effect on more robust models such as GRAPH-PROMPTER, and because textual features contribute substantially to robustness, we evaluate unnoticeable textual feature perturbations as a separate attack channel.

We adopt the imperceptible attack strategies of (Boucher et al. 2022) with one modification: perturbations are generated using a task-agnostic English-to-French translation objective. The optimization modifies input text so that the rendered text remains visually identical or highly similar to the original text, while the translation output becomes incorrect, thereby corrupting the semantics encoded by the model. We instantiate two imperceptible attacks: *Homoglyphs*, which replace characters with visually similar Unicode characters, and *Reorderings*, which use direction-control characters to change rendering order. Examples of Homoglyph and Reordering perturbations are provided in Appendix A.6.

Following (Boucher et al. 2022), we optimize perturbations with differential evolution (Storn and Price 1997). The objective maximizes the Levenshtein distance between model outputs produced from perturbed and unperturbed text. The perturbation budget is fixed at 10% of the average length of each textual feature.

Results. Table 4 shows that Reordering attacks are generally stronger than Homoglyph attacks across models and datasets. On Cora and CiteSeer, LLAGA is vulnerable to both attack types: Homoglyph poisoning reduces accuracy by 27% and 23%, whereas Reordering poisoning reduces accuracy by 35% and 25%, respectively. GRAPH-PROMPTER is more resilient in the corresponding settings. PubMed follows the same qualitative pattern, although the absolute degradation is smaller than on the two smaller datasets.

Comparison with structural perturbations in Figure 3 indicates a shift in the dominant vulnerability mode. Feature perturbations, particularly Reordering, produce substantially larger degradation than structural perturbations from METAATTACK on both LLAGA and GRAPH-PROMPTER. This result suggests that graph-aware LLMs depend strongly on textual node features relative to graph topology, making feature perturbations a critical attack channel. The next section evaluates the joint effect of feature and structural perturbations in a unified attack.

8 A Unified Imperceptible Attack on Graph-aware LLMs

We formulate a unified attack that jointly applies feature and structural perturbations to produce an adversarial input that is both inconspicuous and effective. The hypothesis is that simultaneous perturbation of the textual and relational channels will reduce model accuracy more than either perturbation class alone, while preserving unnoticeable changes to both graph structure and node features.

For the structural component, we use METAATTACK to modify graph topology through edge perturbations. For the feature component, we combine Homoglyph and Reordering perturbations as described in the previous section. The resulting attack targets the relational channel and the textual channel simultaneously, matching the two information sources used by graph-aware LLMs.

Table 4: Accuracy under feature perturbation attacks, with percentage decrease from clean accuracy shown in parentheses.

Data	Method	Poisoning		Evasion	
		LLAGA	GRAPHPROMPTER	LLAGA	GRAPHPROMPTER
Cora	Homoglyph	0.65 ± 0.05 (27%)	0.52 ± 0.04 (13%)	0.41 ± 0.04 (54%)	0.38 ± 0.03 (37%)
	Reordering	0.58 ± 0.06 (35%)	0.45 ± 0.05 (25%)	0.24 ± 0.05 (73%)	0.20 ± 0.03 (67%)
Citeseer	Homoglyph	0.54 ± 0.04 (23%)	0.65 ± 0.04 (7%)	0.32 ± 0.03 (55%)	0.52 ± 0.05 (26%)
	Reordering	0.46 ± 0.03 (35%)	0.39 ± 0.02 (44%)	0.20 ± 0.05 (72%)	0.27 ± 0.02 (61%)
PubMed	Homoglyph	0.72 ± 0.04 (20%)	0.83 ± 0.04 (8%)	0.36 ± 0.06 (60%)	0.55 ± 0.03 (39%)
	Reordering	0.67 ± 0.05 (26%)	0.80 ± 0.03 (11%)	0.28 ± 0.04 (69%)	0.42 ± 0.04 (53%)

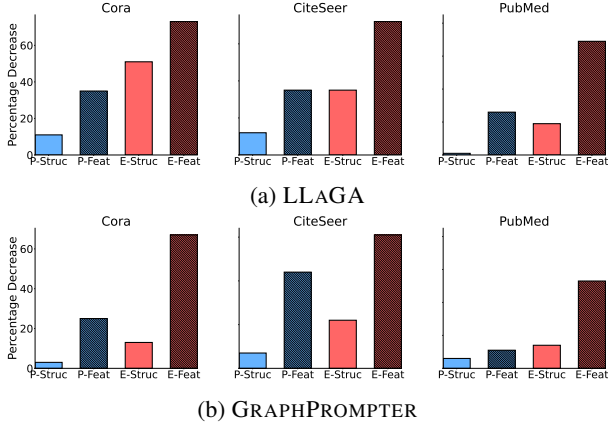


Figure 3: Comparison of structural and feature-based adversarial attacks on graph-aware LLMs. Attack success rate is measured as the percentage decrease from the original accuracy on the y-axis. P-Struc, P-Feat, E-Struc, and E-Feat denote poisoning-structure, poisoning-feature, evasion-structure, and evasion-feature attacks, respectively.

Results. As shown in Table 5, the unified attack, which jointly applies structural and feature perturbations, degrades performance more strongly than either component alone across datasets. This result exposes an architectural property of graph-aware LLMs: unlike traditional GNNs, whose adversarial failure modes are often dominated by structural perturbations, graph-aware LLMs are also highly sensitive to feature-level perturbations. The effectiveness of the unified attack indicates that these systems require consistency between graph topology and textual node features, and that simultaneous low-visibility perturbations to both channels can severely compromise prediction.

9 Ablation Study

9.1 Varying Feature Perturbation

To characterize the relation between perturbation budget and attack effectiveness, we vary the percentage of perturbed features. We analyze the Reordering attack, which follows the same qualitative pattern as the Homoglyph attack, over perturbation rates from 1% to 10%. Across all models and datasets (Cora, Citeseer, and PubMed), reducing the perturbation per-

Table 5: Model accuracy under the unified imperceptible attack, with percentage decrease from clean accuracy shown in parentheses.

Data	Attack	LLAGA	GRAPHPROMPTER
Cora	Poisoning	0.35 ± 0.05 (61%)	0.28 ± 0.08 (53%)
	Evasion	0.14 ± 0.06 (84%)	0.11 ± 0.05 (82%)
Citeseer	Poisoning	0.26 ± 0.04 (63%)	0.22 ± 0.03 (69%)
	Evasion	0.10 ± 0.09 (86%)	0.13 ± 0.07 (81%)
PubMed	Poisoning	0.39 ± 0.07 (57%)	0.32 ± 0.04 (64%)
	Evasion	0.12 ± 0.05 (87%)	0.19 ± 0.04 (79%)

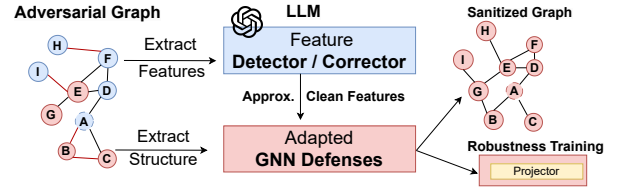


Figure 4: **Overview of the GALGUARD defense pipeline.** Perturbed node features and edges are shown in blue and red, respectively. The LLM corrector detects anomalous textual features extracted from the adversarial graph and produces corrected feature representations. The corrected features, together with the perturbed edge set, are then processed by GNN defense modules, which either produce a sanitized graph or are adapted for robustness training of graph-to-LLM projectors.

centage decreases attack effectiveness, as expected. As the perturbation rate decreases, poisoning and evasion success rates also tend to converge. This convergence occurs because small perturbation budgets produce a weak adversarial signal, limiting model degradation regardless of whether the perturbation is introduced during training (poisoning) or inference (evasion). A detailed plot of these trends is provided in the Appendix (Figure 2).

9.2 Does the Choice of LLM Affect Attack Performance?

We test whether the foundation LLM backbone affects adversarial robustness in graph-aware LLMs. The default configuration uses Vicuna-7B (Chiang et al. 2023) for LLAGA and Llama2-7B (Touvron et al. 2023) for GRAPHPROMPTER. We replace the LLAGA backbone with Llama2-7B and compare clean, poisoning, and evasion performance across datasets. The two backbones produce similar clean accuracy (e.g., Vicuna: 0.89 ± 0.07 , Llama2: 0.90 ± 0.03 on Cora) and comparable attack vulnerability. The same pattern holds when LLM backbones are exchanged in GRAPHPROMPTER. These results indicate that, within the evaluated configurations, foundation-model choice has limited effect on adversarial robustness. Complete results are reported in Table 2 of the Appendix.

10 Defense: GALGUARD

Existing GNN defenses are not directly sufficient for graph-aware LLMs because these models expose coupled structural and textual feature vulnerabilities. Many graph defenses prune or reweight edges using node feature similarity (Wu et al. 2019; Dai et al. 2018; Zhang and Zitnik 2020; Jin et al. 2020; Zhang and Ma 2024), which implicitly assumes that the feature vectors are reliable. When textual features are adversarially perturbed, this assumption is violated and the defense can fail (see Figure 3 in the Appendix). Similarly, methods that decouple structure and features (Shanthamallu, Thiagarajan, and Spanias 2021; Wu et al. 2021), as well as OCR-based sanitization (Boucher et al. 2022), are insufficient under coordinated attacks. These failure modes motivate a defense that jointly addresses feature integrity and structural robustness.

We therefore propose graph-aware LLM defense (GALGUARD), an end-to-end defense framework with two complementary components: (1) an LLM-based feature corrector and (2) adapted GNN-based structural defenses.

Feature Robustness Strategy. As illustrated in Figure 4, the LLM corrector functions as an adaptive text-processing module that inspects node features for anomalous character-level or ordering patterns. It then generates approximate corrections intended to restore feature integrity and reduce the effect of feature-level perturbations. The prompt instructions used by this module are provided in Appendix A.6. In our experiments, GPT-4 Turbo is used as the LLM corrector.

Graph Structure Robustness Strategy. To improve structural robustness, we use a two-stage procedure. First, we apply a *graph purification heuristic* (Wu et al. 2019) as pre-processing: for each node v , we compute the cosine similarity between its embedding $\phi(x_v)$ and each neighbor embedding $\phi(x_u)$, and remove edges whose similarity is low. Second, we incorporate *GNNGuard* (Zhang and Zitnik 2020). For GRAPHPROMPTER, GNNGuard is applied directly to the message-passing layers. For LLAGA, we adapt the defense by inserting purification into the construction of the computational tree used for node sequence generation. We also introduce a learnable *global structural context embedding* (M_{global}) as a parameter of the LLAGA projector. By concatenating M_{global} with the purified node sequence embedding and training them jointly, the projector learns to operate on graph inputs that may contain residual perturbations. Because GALGUARD is modular, alternative structural defenses such as Pro-GNN (Jin et al. 2020) can replace GNNGuard.

10.1 Results

Table 6 reports accuracy under METAATTACK and under two defense configurations: GALGUARD_p, which combines feature correction with graph purification, and the full GALGUARD, which adds GNNGuard or its LLAGA-specific adaptation. Across Cora, CiteSeer, and PubMed, and under both poisoning and evasion attacks, the proposed defenses consistently reduce the effect of METAATTACK. GALGUARD_p provides a strong intermediate defense, showing that LLM-based feature correction plus initial purification improves robustness. GALGUARD further improves accuracy in all re-

ported settings, indicating that structural defense and feature correction provide complementary protection.

Table 6: Accuracy under METAATTACK and the proposed GALGUARD variants for poisoning and evasion attacks. GALGUARD_p combines feature correction with graph purification, and GALGUARD_p is evaluated as the intermediate defense configuration.

Data	Method	LLAGA		GRAPHPROMPTER	
		Poisoning	Evasion	Poisoning	Evasion
Cora	METAATTACK	0.79 $\pm$ 0.03	0.44 $\pm$ 0.06	0.58 $\pm$ 0.05	0.52 $\pm$ 0.04
	GALGUARD _p	0.82 $\pm$ 0.05	0.62 $\pm$ 0.04	0.59 $\pm$ 0.03	0.56 $\pm$ 0.04
	GALGUARD	0.85$\pm$0.04	0.83$\pm$0.04	0.60$\pm$0.05	0.59$\pm$0.03
CiteSeer	MetaAttack	0.63 $\pm$ 0.04	0.50 $\pm$ 0.03	0.65 $\pm$ 0.06	0.56 $\pm$ 0.03
	GALGUARD _p	0.64 $\pm$ 0.03	0.56 $\pm$ 0.02	0.67 $\pm$ 0.05	0.60 $\pm$ 0.06
	GALGUARD	0.67$\pm$0.04	0.62$\pm$0.04	0.68$\pm$0.04	0.64$\pm$0.05
PubMed	MetaAttack	0.89 $\pm$ 0.08	0.73 $\pm$ 0.04	0.85 $\pm$ 0.02	0.77 $\pm$ 0.03
	GALGUARD _p	0.90 $\pm$ 0.06	0.78 $\pm$ 0.03	0.85 $\pm$ 0.04	0.81 $\pm$ 0.04
	GALGUARD	0.90$\pm$0.01	0.87$\pm$0.02	0.89$\pm$0.02	0.89$\pm$0.01

11 Conclusion

This paper provides an initial systematic characterization of adversarial vulnerability in graph-aware LLMs. The results show that the graph integration mechanism is itself a critical component of the attack surface. In particular, the node sequence template increases susceptibility by exposing placeholder positions that can be exploited through node injection. The experiments further show that imperceptible textual feature perturbations are highly effective against graph-aware LLMs, often exceeding the impact of traditional structural attacks, and that a unified attack combining feature and structural perturbations can reduce performance to near-useless levels.

To mitigate these vulnerabilities, we introduce GALGUARD, an end-to-end defense framework that integrates an LLM-based feature corrector with adapted GNN-based structural defenses. As graph-aware LLMs become candidates for deployment in domains such as social networks, healthcare, and finance, their resilience to adversarial manipulation becomes a necessary reliability requirement. We expect this study to support further work on formal threat models, attack-specific defenses, and deployment-oriented safeguards for graph-aware LLM systems.

References

- Achiam, J.; Adler, S.; Agarwal, S.; Ahmad, L.; Akkaya, I.; Aleman, F. L.; Almeida, D.; Altenschmidt, J.; Altman, S.; Anadkat, S.; et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.
- Ahmedt-Aristizabal, D.; Armin, M. A.; Denman, S.; Fookes, C.; and Petersson, L. 2021. Graph-based deep learning for medical diagnosis and analysis: past, present and future. *Sensors*, 21(14): 4758.
- Anil, R.; Dai, A. M.; Firat, O.; Johnson, M.; Lepikhin, D.; Passos, A.; Shakeri, S.; Taropa, E.; Bailey, P.; Chen,

- Z.; et al. 2023. Palm 2 technical report. *arXiv preprint arXiv:2305.10403*.
- Bi, B.; Liu, S.; Wang, Y.; Mei, L.; and Chen, X. 2024. Scalable Link Prediction on Large-Scale Heterogeneous Graphs with Large Language Models. *arXiv preprint arXiv:2401.13227*.
- Boucher, N.; Shumailov, I.; Anderson, R.; and Papernot, N. 2022. Bad characters: Imperceptible nlp attacks. In *2022 IEEE Symposium on Security and Privacy (SP)*, 1987–2004. IEEE.
- Brannon, W.; Fulay, S.; Jiang, H.; Kang, W.; Roy, B.; Kabbara, J.; and Roy, D. 2023. Congrat: Self-supervised contrastive pretraining for joint graph and text embeddings. *arXiv preprint arXiv:2305.14321*.
- Chai, Z.; Zhang, T.; Wu, L.; Han, K.; Hu, X.; Huang, X.; and Yang, Y. 2023. GraphLLM: Boosting Graph Reasoning Ability of Large Language Model. *arXiv preprint arXiv:2310.05845*.
- Chandra, S.; Mishra, P.; Yannakoudakis, H.; Nimishakavi, M.; Saeidi, M.; and Shutova, E. 2020. Graph-based Modeling of Online Communities for Fake News Detection. *arXiv preprint arXiv:2008.06274*.
- Chen, R.; Zhao, T.; Jaiswal, A.; Shah, N.; and Wang, Z. 2024. LLaGA: Large Language and Graph Assistant. In *Forty-first International Conference on Machine Learning*.
- Chen, Z.; Mao, H.; Li, H.; Jin, W.; Wen, H.; Wei, X.; Wang, S.; Yin, D.; Fan, W.; Liu, H.; and Tang, J. 2023a. Exploring the Potential of Large Language Models (LLMs) in Learning on Graphs. *arXiv preprint arXiv:2307.03393*.
- Chen, Z.; Mao, H.; Wen, H.; Han, H.; Jin, W.; Zhang, H.; Liu, H.; and Tang, J. 2023b. Label-free Node Classification on Graphs with Large Language Models (LLMs). *arXiv preprint arXiv:2310.04668*.
- Chiang, W.-L.; Li, Z.; Lin, Z.; Sheng, Y.; Wu, Z.; Zhang, H.; Zheng, L.; Zhuang, S.; Zhuang, Y.; Gonzalez, J. E.; Stoica, I.; and Xing, E. P. 2023. Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90%* ChatGPT Quality.
- Conti, M.; Li, J.; Picek, S.; and Xu, J. 2022. Label-only membership inference attack against node-level graph neural networks. In *Proceedings of the 15th ACM Workshop on Artificial Intelligence and Security*, 1–12.
- Dai, E.; Lin, M.; Zhang, X.; and Wang, S. 2023. Unnoticeable backdoor attacks on graph neural networks. In *Proceedings of the ACM Web Conference 2023*, 2263–2273.
- Dai, H.; Li, H.; Tian, T.; Huang, X.; Wang, L.; Zhu, J.; and Song, L. 2018. Adversarial attack on graph structured data. In *International conference on machine learning*, 1115–1124. PMLR.
- Dong, N.; Mücke, S.; and Khosla, M. 2022. Mucomid: A multitask graph convolutional learning framework for miRNA-disease association prediction. *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, 19(6): 3081–3092.
- Duan, K.; Liu, Q.; Chua, T.-S.; Yan, S.; Ooi, W. T.; Xie, Q.; and He, J. 2023. SimTeG: A Frustratingly Simple Approach Improves Textual Graph Learning. *arXiv preprint arXiv:2308.02565*.
- Dubey, A.; Jauhri, A.; Pandey, A.; Kadian, A.; Al-Dahle, A.; Letman, A.; Mathur, A.; Schelten, A.; Yang, A.; Fan, A.; et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Duddu, V.; Boutet, A.; and Shejwalkar, V. 2020. Quantifying privacy leakage in graph embedding. In *MobiQuitous 2020-17th EAI International Conference on Mobile and Ubiquitous Systems: Computing, Networking and Services*, 76–85.
- Fan, W.; Ma, Y.; Li, Q.; He, Y.; Zhao, E.; Tang, J.; and Yin, D. 2019. Graph neural networks for social recommendation. In *The world wide web conference*, 417–426.
- Fatemi, B.; Halcrow, J.; and Perozzi, B. 2023. Talk like a Graph: Encoding Graphs for Large Language Models. *arXiv preprint arXiv:2310.04560*.
- Gong, N. Z.; and Liu, B. 2018. Attribute inference attacks in online social networks. *ACM Transactions on Privacy and Security (TOPS)*, 21(1): 1–30.
- Guo, J.; Du, L.; Liu, H.; Zhou, M.; He, X.; and Han, S. 2023. GPT4Graph: Can Large Language Models Understand Graph Structured Data ? An Empirical Evaluation and Benchmarking. *arXiv preprint arXiv:2305.15066*.
- Hamilton, W.; Ying, Z.; and Leskovec, J. 2017. Inductive representation learning on large graphs. *Advances in neural information processing systems*, 30.
- He, X.; Bresson, X.; Laurent, T.; Perold, A.; LeCun, Y.; and Hooi, B. 2024. Harnessing Explanations: LLM-to-LM Interpreter for Enhanced Text-Attributed Graph Representation Learning. In *The Twelfth International Conference on Learning Representations*.
- He, X.; Wen, R.; Wu, Y.; Backes, M.; Shen, Y.; and Zhang, Y. 2021. Node-level membership inference attacks against graph neural networks. *arXiv preprint arXiv:2102.05429*.
- Hu, W.; Fey, M.; Zitnik, M.; Dong, Y.; Ren, H.; Liu, B.; Catasta, M.; and Leskovec, J. 2020. Open graph benchmark: Datasets for machine learning on graphs. *Advances in neural information processing systems*, 33: 22118–22133.
- Huang, J.; Zhang, X.; Mei, Q.; and Ma, J. 2023a. Can LLMs Effectively Leverage Structural Information for Graph Learning: When and Why. *arXiv preprint arXiv:2309.16595*.
- Huang, X.; Han, K.; Bao, D.; Tao, Q.; Zhang, Z.; Yang, Y.; and Zhu, Q. 2023b. Prompt-based Node Feature Extractor for Few-shot Learning on Text-Attributed Graphs. *arXiv preprint arXiv:2309.02848*.
- Jin, W.; Li, Y.; Xu, H.; Wang, Y.; Ji, S.; Aggarwal, C.; and Tang, J. 2021. Adversarial attacks and defenses on graphs. *ACM SIGKDD Explorations Newsletter*, 22(2): 19–34.
- Jin, W.; Ma, Y.; Liu, X.; Tang, X.; Wang, S.; and Tang, J. 2020. Graph structure learning for robust graph neural networks. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, 66–74.
- Kipf, T. N.; and Welling, M. 2022. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations*.
- Kuang, H.; Xu, J.; Zhang, H.; Zhao, Z.; Zhang, Q.; Huang, X.; and Wei, Z. 2023. Unleashing the Power of Language Models

- in Text-Attributed Graph. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 8429–8441. Association for Computational Linguistics.
- Li, K.; Liu, Y.; Ao, X.; and He, Q. 2023. Revisiting graph adversarial attack and defense from a data distribution perspective. In *The Eleventh International Conference on Learning Representations*.
- Liu, C.; and Wu, B. 2023. Evaluating Large Language Models on Graphs: Performance Insights and Comparative Analysis. *arXiv preprint arXiv:2308.11224*.
- Liu, S.; Nie, W.; Wang, C.; Lu, J.; Qiao, Z.; Liu, L.; Tang, J.; Xiao, C.; and Anandkumar, A. 2023. Multi-modal molecule structure–text model for text-based retrieval and editing. *Nature Machine Intelligence*, 5(12): 1447–1457.
- Liu, Z.; He, X.; Tian, Y.; and Chawla, N. V. 2024. Can we soft prompt LLMs for graph learning tasks? In *Companion Proceedings of the ACM on Web Conference 2024*, 481–484.
- Ma, J.; Ding, S.; and Mei, Q. 2020. Towards more practical adversarial attacks on graph neural networks. *Advances in neural information processing systems*, 33: 4756–4766.
- Mavromatis, C.; Ioannidis, V. N.; Wang, S.; Zheng, D.; Adeshina, S.; Ma, J.; Zhao, H.; Faloutsos, C.; and Karypis, G. 2023. Train your own GNN teacher: Graph-aware distillation on textual graphs. *arXiv preprint arXiv:2304.10668*.
- Olatunji, I. E.; Hizber, A.; Sihlovec, O.; and Khosla, M. 2023a. Does black-box attribute inference attacks on graph neural networks constitute privacy risk? *arXiv preprint arXiv:2306.00578*.
- Olatunji, I. E.; Nejdil, W.; and Khosla, M. 2021. Membership inference attack on graph neural networks. In *2021 Third IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications (TPS-ISA)*, 11–20. IEEE.
- Olatunji, I. E.; Rathee, M.; Funke, T.; and Khosla, M. 2023b. Private Graph Extraction via Feature Explanations. *Proceedings on Privacy Enhancing Technologies*.
- Pan, S.; Luo, L.; Wang, Y.; Chen, C.; Wang, J.; and Wu, X. 2024. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*.
- Peng, Y.; Lin, S.; Chen, Q.; Xu, L.; Ren, X.; Li, Y.; and Xu, J. 2024. ChatGraph: Chat with Your Graphs. *arXiv preprint arXiv:2401.12672*.
- Qin, Y.; Wang, X.; Zhang, Z.; and Zhu, W. 2023. Disentangled Representation Learning with Large Language Models for Text-Attributed Graphs. *arXiv preprint arXiv:2310.18152*.
- Shanthamallu, U. S.; Thiagarajan, J. J.; and Spanias, A. 2021. Uncertainty-matching graph neural networks to defend against poisoning attacks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, 9524–9532.
- Storn, R.; and Price, K. 1997. Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces. *Journal of global optimization*, 11: 341–359.
- Su, B.; Du, D.; Yang, Z.; Zhou, Y.; Li, J.; Rao, A.; Sun, H.; Lu, Z.; and Wen, J.-R. 2022. A molecular multimodal foundation model associating molecule graphs with natural language. *arXiv preprint arXiv:2209.05481*.
- Sun, L.; Dou, Y.; Yang, C.; Zhang, K.; Wang, J.; Philip, S. Y.; He, L.; and Li, B. 2022. Adversarial attack and defense on graph data: A survey. *IEEE Transactions on Knowledge and Data Engineering*, 35(8): 7693–7711.
- Sun, S.; Ren, Y.; Ma, C.; and Zhang, X. 2023. Large Language Models as Topological Structure Enhancers for Text-Attributed Graphs. *arXiv preprint arXiv:2311.14324*.
- Tan, Y.; Zhou, Z.; Lv, H.; Liu, W.; and Yang, C. 2023. WalkLM: A Uniform Language Model Fine-tuning Framework for Attributed Graph Embedding. In *Thirty-seventh Conference on Neural Information Processing Systems*.
- Team, G.; Anil, R.; Borgeaud, S.; Wu, Y.; Alayrac, J.-B.; Yu, J.; Soricut, R.; Schalkwyk, J.; Dai, A. M.; Hauth, A.; et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*.
- Tian, Y.; Song, H.; Wang, Z.; Wang, H.; Hu, Z.; Wang, F.; Chawla, N. V.; and Xu, P. 2024. Graph neural prompting with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 19080–19088.
- Touvron, H.; Martin, L.; Stone, K.; Albert, P.; Almahairi, A.; Babaei, Y.; Bashlykov, N.; Batra, S.; Bhargava, P.; Bhosale, S.; et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.
- Veličković, P.; Cucurull, G.; Casanova, A.; Romero, A.; Liò, P.; and Bengio, Y. 2018. Graph Attention Networks. In *International Conference on Learning Representations*.
- Wang, J.; Zhang, S.; Xiao, Y.; and Song, R. 2021. A review on graph neural network methods in financial applications. *arXiv preprint arXiv:2111.15367*.
- Wang, X.; and Wang, W. H. 2022. Group property inference attacks against graph neural networks. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2871–2884.
- Wei, Y.; Fu, S.; Jiang, W.; Kwok, J. T.; and Zhang, Y. 2024a. Rendering graphs for graph reasoning in multimodal large language models. *arXiv preprint arXiv:2402.02130*.
- Wei, Y.; Huang, Q.; Kwok, J. T.; and Zhang, Y. 2024b. Kicgpt: Large language model with knowledge in context for knowledge graph completion. *arXiv preprint arXiv:2402.02389*.
- Wenkel, F.; Wolf, G.; and Knyazev, B. 2023. Pretrained Language Models to Solve Graph Tasks in Natural Language. In *ICML workshop on Structured Probabilistic Inference & Generative Modeling*.
- Wu, H.; Wang, C.; Tyshetskiy, Y.; Docherty, A.; Lu, K.; and Zhu, L. 2019. Adversarial examples on graph data: Deep insights into attack and defense. *arXiv preprint arXiv:1903.01610*.
- Wu, X.; Wu, H.; Zhou, X.; and Lu, K. 2021. CoG: a Two-View Co-training Framework for Defending Adversarial Attacks on Graph. *arXiv preprint arXiv:2109.05558*.
- Xie, H.; Zheng, D.; Ma, J.; Zhang, H.; Ioannidis, V. N.; Song, X.; Ping, Q.; Wang, S.; Yang, C.; Xu, Y.; et al. 2023. Graph-aware language model pre-training on a large graph corpus can help multiple graph applications. In *Proceedings of the*

29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 5270–5281.

Yang, J.; Liu, Z.; Xiao, S.; Li, C.; Lian, D.; Agrawal, S.; Singh, A.; Sun, G.; and Xie, X. 2021. GraphFormers: GNN-nested Transformers for Representation Learning on Textual graph. In *Proceedings of the 35th Conference on Neural Information Processing Systems*.

Yang, Z.; Cohen, W.; and Salakhudinov, R. 2016. Revisiting semi-supervised learning with graph embeddings. In *International conference on machine learning*, 40–48. PMLR.

Ye, R.; Zhang, C.; Wang, R.; Xu, S.; and Zhang, Y. 2023. Natural Language is All a Graph Needs. *arXiv preprint arXiv:2308.07134*.

Zhang, A.; and Ma, J. 2024. Defensevgae: Defending against adversarial attacks on graph data via a variational graph autoencoder. In *International Conference on Intelligent Computing*, 313–324. Springer.

Zhang, J. 2023. Graph-toolformer: To empower llms with graph reasoning ability via prompt augmented by chatgpt. *arXiv preprint arXiv:2304.11116*.

Zhang, X.; and Zitnik, M. 2020. Gnn-guard: Defending graph neural networks against adversarial attacks. *Advances in neural information processing systems*, 33: 9263–9275.

Zhang, Z.; Chen, M.; Backes, M.; Shen, Y.; and Zhang, Y. 2022. Inference attacks against graph neural networks. In *31st USENIX Security Symposium (USENIX Security 22)*, 4543–4560.

Zhang, Z.; Liu, Q.; Huang, Z.; Wang, H.; Lu, C.; Liu, C.; and Chen, E. 2021. Graphmi: Extracting private graph data from graph neural networks. *arXiv preprint arXiv:2106.02820*.

Zhang, Z.; Wang, X.; Zhang, Z.; Li, H.; Qin, Y.; and Zhu, W. 2023. LLM4DyG: Can Large Language Models Solve Problems on Dynamic Graphs? *arXiv preprint arXiv:2310.17110*.

Zhao, J.; Zhuo, L.; Shen, Y.; Qu, M.; Liu, K.; Bronstein, M.; Zhu, Z.; and Tang, J. 2023. GraphText: Graph Reasoning in Text Space. *arXiv preprint arXiv:2310.01089*.

Zhu, Y.; Wang, Y.; Shi, H.; and Tang, S. 2024. Efficient tuning and inference for large language models on textual graphs. *arXiv preprint arXiv:2401.15569*.

Zügner, D.; Akbarnejad, A.; and Günnemann, S. 2018. Adversarial attacks on neural networks for graph data. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, 2847–2856.

Zügner, D.; and Günnemann, S. 2019. Adversarial Attacks on Graph Neural Networks via Meta Learning. In *International Conference on Learning Representations (ICLR)*.

A Appendix

Organization: The Appendix is structured as follows. Appendix A.1 provides a detailed taxonomy of graph-aware LLMs. Appendix A.2 reviews related work on attacks against Graph Neural Networks (GNNs). Appendix A.3 specifies the adversarial procedures used by NETTACK and METAATTACK, and Appendix A.4 gives the node sequence injection algorithm for LLAGA. Appendix A.5 reports ablation results for feature perturbation rate and base-LLM choice. Appendix A.7 provides feature-attack examples and the LLM corrector template, and Appendix A.8 discusses implications of the empirical findings.

Code. Our code will be released upon acceptance.

A.1 Graph Encoding-Based Taxonomy of LLM Adaptations

The integration of large language models (LLMs) with graph data has produced several adaptation mechanisms for graph reasoning, representation learning, and inference. These mechanisms can be organized by the graph encoding supplied to the LLM: *textual descriptions of graphs*, where graph information is represented as natural language, and *learned graph projector*, where a trainable projector maps graph data into embeddings or structured forms that the LLM can process. This section specifies the methods in each encoding class. Figure 5 illustrates both classes, and Table 7 summarizes their operational differences.

Graph Representation as a Textual Description In the textual-description category, graph data is serialized into natural language, enabling the LLM to receive graph structure through the prompt interface. Prior work (Guo et al. 2023; Liu and Wu 2023; Ye et al. 2023; Wenkel, Wolf, and Knyazev 2023; Wei et al. 2024b; Fatemi, Halcrow, and Perozzi 2023; Tan et al. 2023; Zhao et al. 2023; Wei et al. 2024a) converts graph data into textual formats that are often paired with task-specific query instructions. For example, Tan et al. (2023) introduced a unified attributed-graph embedding framework that generates random-walk textual sequences from graphs and uses them to fine-tune LLMs. Similarly, Zhao et al. (2023) studied graph reasoning in text space by translating graph structures into natural language. Extending this direction, Wei et al. (2024a) augmented textual graph representations with visual graph representations to improve LLM graph reasoning. In multimodal applications, Liu et al. (2023) and Su et al. (2022) associated molecular graphs with textual descriptions for drug discovery and molecular retrieval. In addition, Huang et al. (2023a) examined whether incorporating graph structure into LLM prompts improves predictive performance, finding that LLMs tend to process graph prompts as contextual paragraphs rather than explicit graph structures. This behavior makes prompt design technically important, as also reflected in studies that extend LLMs to heterogeneous and large-scale graph tasks (Brannon et al. 2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024). Complementary work (He et al. 2024) uses LLM-generated explanations as graph features to improve both performance and interpretability.

A related line of work preserves textual graph representations while integrating pretraining or GNN components to improve graph reasoning. For instance, Chen et al. (2023a) proposed two LLM-GNN integration strategies: using LLMs to enrich node features for node classification and using LLMs as direct predictors for graph tasks. Similarly, Chandra et al. (2020) used LLMs as text encoders and GNNs as structure encoders for fake-news detection in online communities. Chen et al. (2023b) studied node classification without explicit labels by using LLMs to annotate nodes before training a GNN. Likewise, Mavromatis et al. (2023) used an LLM as a feature extractor for GNN models.

For inference, Zhu et al. (2024) proposed a fine-tuning method for textual graphs that combines LLMs and GNNs through a tunable side structure attached to each LLM layer, improving training efficiency and inference speed. Duan et al. (2023) introduced SimTeG, which improves textual graph learning with LLM representations: the LLM is first fine-tuned on a downstream task such as node classification, and node embeddings are then obtained from the final hidden states. Additionally, Zhang (2023) proposed Graph-toolformer, which uses ChatGPT-powered prompt engineering to augment graph reasoning capabilities.

Although textual-description methods provide a direct interface between LLMs and graph tasks, they may fail to preserve fine-grained relational structure. Their performance is also highly sensitive to prompt specification, requiring careful design and empirical tuning to obtain strong results.

Graph Representation as a Learned Projector Learned-projector methods address limitations of textual serialization by transforming graph data into embeddings or graph vectors that can be supplied to the LLM more directly. For instance, Chen et al. (2024) introduced LLAGA, a large language and graph assistant that preserves the general-purpose LLM while converting graph structure into graph tokens compatible with LLM processing. Similarly, (Liu et al. 2024; Qin et al. 2023) incorporated graph-structure information into a pre-trained LLM through tailored disentangled graph neural network layers. Tian et al. (2024) proposed graph neural prompts for encoding knowledge graphs into graph embeddings used during inference. Huang et al. (2023b) introduced G-Prompt, a prompt-based node feature extractor for few-shot learning on text-attributed graphs, which combines a learnable GNN adapter with task-specific prompts to capture neighborhood information and extract informative node features.

Another line of work compresses graph information into fixed-length prefixes using graph transformers attached to LLM layers, thereby augmenting reasoning capacity (Chai et al. 2023). Yang et al. (2021) developed GraphFormers, which integrates text encoding and graph aggregation by nesting GNNs alongside transformer blocks in an iterative workflow. Peng et al. (2024) developed ChatGraph, an interactive natural-language interface for graph analysis in which graphs are represented as sequential paths and the LLM is fine-tuned on graph data.

We exclude projector adaptations that require retraining the LLM, because they substantially increase computational

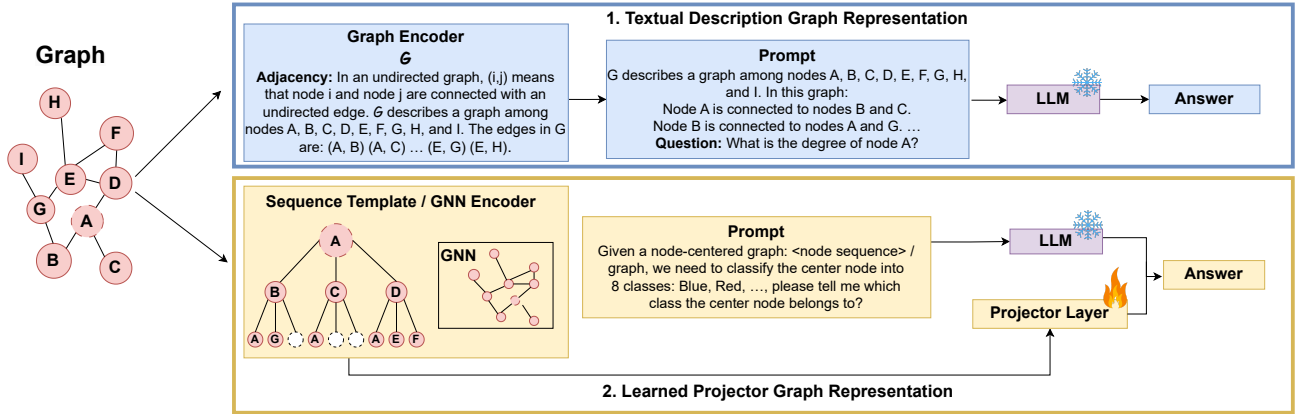


Figure 5: **Graph encoding-based adaptations of LLMs.** In the textual-description representation, graph information is serialized as text (top). In the learned-projector representation, graph information is encoded either by node templates that convert neighborhoods into sequences or by GNNs (bottom). In both settings, the LLM remains frozen and only the projector that maps graph representations into tokens is trained.

Table 7: Technical comparison of graph encoding-based LLM adaptations.

Aspect	Textual Description Approach	Learned Projector Approach
Graph Encoding	Converts graph data (nodes, edges) into natural language descriptions.	Uses a learned projector to encode graph structures into embeddings or graph tokens.
LLM Modifications	No modification to the LLM.	No modification to the LLM.
Graph Structure Representation	Describes graph topology and relationships using textual narratives.	Encodes relational and structural information directly into vector representations.
Flexibility	Can only be used for simple graphs.	More flexible in representing complex graph structures.
Computational Cost	No additional training is required. Lower cost.	Lightweight projector is trained, typically a linear layer. Minimal cost.
Suitability for Graph Tasks	May lose some relational nuance due to limitations in textual descriptions.	Captures detailed structural information, improving performance on graph-specific tasks.

cost without providing commensurate performance gains in the settings considered here. Instead, we focus on methods that use **frozen pre-trained LLMs** (Chen et al. 2024; Liu et al. 2024; Qin et al. 2023; Tian et al. 2024; Huang et al. 2023b). Freezing the LLM reduces optimization cost, improves compatibility with multiple backbones, and preserves the pretrained model’s general knowledge while still allowing graph-task adaptation through the projector. Throughout this work, we evaluate two representative projector methods: **LLAGA** (Chen et al. 2024), which uses node-level templates and a linear projector, and **GRAPHPROMPTER** (Liu et al. 2024), which uses a GNN-based projector to encode graph information.

This work. We do not evaluate textual-description methods because they often cannot represent detailed graph structure at scale. Such methods are generally suitable for simpler graph properties, such as node degree, but become impractical for larger graphs when prompt token limits constrain the amount of topology that can be serialized. From an adversarial perspective, textual-description methods also do not introduce the projector-specific attack surfaces examined in this paper. When graph structure is represented as ordinary text (e.g., “Node A is connected to Node B”), the resulting model is primarily exposed to conventional text-based adversarial attacks rather than graph-specific perturbations of structure or textual node features. We therefore focus on projector-based graph-aware LLMs, where the graph-to-token interface creates distinct robustness questions.

A.2 Attacks on GNN

GNNs are subject to several attack classes. Membership inference attacks (Olatunji, Nejd, and Khosla 2021; He et al. 2021; Conti et al. 2022) infer whether a specific node appeared in the training set. Attribute inference attacks (Gong and Liu 2018; Olatunji et al. 2023a; Duddu, Boutet, and Shejwalkar 2020) use model outputs or learned embeddings to infer sensitive or missing node attributes. Property inference attacks (Zhang et al. 2022; Wang and Wang 2022) extract global properties of the graph. Adversarial attacks (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Li et al. 2023; Dai et al. 2018; Sun et al. 2022) degrade model performance through small perturbations to graph structure or node features. Graph reconstruction attacks (Zhang et al. 2021; Olatunji et al. 2023b) attempt to reconstruct the underlying graph from embeddings or model outputs.

Among these categories, adversarial attacks are central to this work because they directly reduce predictive performance. In these attacks, the adversary introduces small, carefully selected *perturbations*, such as node-feature modifications or edge additions/deletions, with the objective of degrading GNN accuracy. The attacks are typically divided into **poisoning attacks**, which manipulate the training data during the *training phase*, and **evasion attacks**, which perturb the input graph at *test time*. For example, Zügner, Akbarnejad, and Günnemann (2018) showed that perturbing graph structure or node features can substantially degrade GNN performance. Their method, **NETTACK**, uses greedy optimization to identify minimal perturbations that maxi-

mize the model’s training loss and disrupt predictions. Li et al. (2023) revisited graph adversarial attacks from a data-distribution perspective, showing that adversarial examples can shift the underlying distribution and proposing a defense mechanism for that effect. Wu et al. (2019) provided a broad analysis of graph adversarial examples and defenses, emphasizing small perturbations and methods for detecting or counteracting them. **METAATTACK** (Zügner and Günnemann 2019) performs poisoning and evasion attacks on GNNs by treating graph structure as an optimizable hyperparameter. It modifies the graph through gradient descent under a bi-level optimization formulation. The method uses meta-gradients to estimate how small structural perturbations affect attacker loss after training, enabling graph updates that reduce model performance.

In this paper, we use **NETTACK** (Zügner, Akbarnejad, and Günnemann 2018) and **METAATTACK** (Zügner and Günnemann 2019) as representative adversarial attack procedures covering both poisoning and evasion settings. Poisoning attacks target model integrity during training, whereas evasion attacks compromise predictions at inference or test time.

A.3 How NETTACK and METAATTACK Work

NETTACK perturbs graph structure by adding or removing edges while preserving the degree distribution of the perturbed graph. It uses greedy optimization to identify minimal perturbations that maximize the training loss of a surrogate model. The surrogate is a linearized 2-layer graph convolutional network (GCN), which approximates the effect of structural changes on classification. The objective of **NETTACK** is to increase confidence in incorrect labels by maximizing the logit discrepancy between the perturbed graph and the clean graph. Operationally, **NETTACK** enumerates candidate edge perturbations around a target node and evaluates a score for each connected edge; this score estimates the effect of modifying that edge on the target-node prediction. The highest-scoring edge is then added or removed. Because this attack is inherently targeted, i.e., it causes misclassification of a specific node, we follow (Zügner and Günnemann 2019) and convert it into an untargeted evaluation by randomly selecting test nodes, attacking them with **NETTACK** while considering all nodes in the graph, and aggregating the results over the attacked test nodes.

METAATTACK optimizes graph structure directly by treating the adjacency matrix as a hyperparameter and applying gradient-based updates. The attack is formulated as a bi-level optimization problem: the inner problem minimizes training loss on the original graph, whereas the outer problem optimizes loss on the poisoned graph. As in **NETTACK**, **METAATTACK** uses a linearized two-layer GCN surrogate to approximate how structural changes affect model performance. A score function evaluates candidate perturbations, and meta-gradient signs with respect to hyperparameters identify connected edges that should be removed. The algorithm then greedily selects edge additions or removals with the highest scores to maximize attack impact.

For both methods, we impose the unnoticeability constraint introduced by (Zügner, Akbarnejad, and Günnemann 2018),

which limits changes to the graph degree distribution. Accordingly, the perturbation budget is fixed at 10% of edges.

In poisoning experiments with LLAGA, the graph perturbed by NETTACK or METAATTACK is used to generate node-level sequences, after which the projector that maps these sequences to token embeddings is retrained. For GRAPH_PROMPTER, the perturbed graph is re-encoded with a graph attention network (GAT), and the projector is re-trained on the resulting graph representations. In evasion experiments, the graph-aware LLM is trained on unperturbed data and perturbations are introduced only during inference.

A.4 Algorithm for Node Sequence Template Injection Attacks

Algorithm 1 specifies the node sequence template injection attack for LLAGA. The model constructs a *fixed-shape* computational tree for each node u by sampling k neighbors per node up to depth d (lines 1–14). If a node has fewer than k neighbors, placeholder entries are inserted, which creates an attack surface. The attack exploits this mechanism by using graph manipulation to place malicious nodes in placeholder positions. We define three strategies: *Non-Adjacent Injection (NI)* inserts nodes outside the 2-hop neighborhood of u (lines 21–27); *Supernode Injection (SI)* inserts one high-degree node into a placeholder (lines 29–33); and *Multiple Supernode Injection (MSI)* distributes distinct high-degree nodes across all placeholders (lines 35–43). These uninformative nodes corrupt the target node sequence and impair training of the projection layer.

A.5 Ablation Studies

This section reports the ablation studies referenced in the main paper.

Ablation Results: Varying Feature Perturbation Figure 6 reports the accuracy of LLAGA and GRAPH_PROMPTER as the feature perturbation percentage varies. Increasing the proportion of perturbed features consistently decreases accuracy for both models, indicating sensitivity to feature-level attacks.

Ablation Results: Does the Choice of LLM Affect Attack Performance? The results in Table 8 indicate that base-LLM choice has limited effect on clean accuracy and adversarial vulnerability within the LLAGA framework. Across Cora, Citeseer, and PubMed, Vicuna-7B and Llama2-7B variants achieve nearly identical clean accuracy and exhibit similar degradation under poisoning and evasion attacks. For example, under METAATTACK evasion on Cora, accuracy is 0.44 ± 0.06 with Vicuna-7B and 0.43 ± 0.08 with Llama2-7B, a difference within the reported standard deviations. The same pattern holds across datasets and attack types, indicating that attack effectiveness is largely independent of the base LLM in these configurations.

A.6 Results on the ArXiv Dataset

We additionally evaluate the attack on the large-scale ArXiv dataset, which contains 169,343 nodes, 1,166,243 edges, and sparsity 0.81. This experiment tests the effectiveness of node

Algorithm 1: Injection Attacks on Neighborhood Detail Template for Obtaining Node Sequence.

Input: Graph $G = (V, E)$, target node u , tree depth d , children size k

Output: Modified graph G' with injected nodes

```

1: procedure NEIGHBORHOODDETAILTEMPLATE( $u, d, k$ )
2:    $T \leftarrow \emptyset$  ▷ Initialize empty tree
3:    $T.\text{root} \leftarrow u$ 
4:   for level  $\ell = 1$  to  $d$  do
5:     for each node  $v$  at level  $\ell - 1$  do
6:        $N_v \leftarrow \text{Sample}(\mathcal{N}(v), k)$  ▷ Sample  $k$  neighbors
7:       if  $|N_v| < k$  then
8:         Fill remaining slots with placeholders
9:       end if
10:      Add  $N_v$  as children of  $v$  in  $T$ 
11:    end for
12:  end for
13:  return  $T$ 
14: end procedure

15: procedure ATTACK( $G, u, d, k, \text{AttackStrategy}$ )
16:    $G' \leftarrow G$  ▷ Create copy of original graph
17:    $T \leftarrow \text{NEIGHBORHOODDETAILTEMPLATE}(u, d, k)$ 
18:    $P \leftarrow \text{GetPlaceholders}(T)$ 
19:   if  $|P| > 0$  then
20:      $\mathcal{N}_2 \leftarrow \text{2-hop neighborhood of } u$ 
21:     if  $\text{AttackStrategy} = \text{NI}$  then
22:        $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
23:        $V_{\text{inject}} \leftarrow \text{Sample}(V_{\text{non-adjacent}}, |P|)$ 
24:       for each placeholder position  $p \in P$  do
25:          $v \leftarrow \text{Next node from } V_{\text{inject}}$ 
26:         Add edge  $(\text{parent}(p), v)$  to  $G'$ 
27:       end for
28:     else if  $\text{AttackStrategy} = \text{SI}$  then
29:        $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
30:        $s \leftarrow \text{GetHighestDegreeNode}(V_{\text{non-adjacent}})$ 
31:       for first placeholder position  $p \in P$  do
32:         Add edge  $(\text{parent}(p), s)$  to  $G'$ 
33:       break ▷ Only use first placeholder
34:     end for
35:     else if  $\text{AttackStrategy} = \text{MSI}$  then
36:        $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
37:        $S \leftarrow \text{GetNodesOrderedByDegree}(V_{\text{non-adjacent}})$ 
38:       for  $i = 1$  to  $|P|$  do
39:          $s \leftarrow \text{Get } i\text{-th node from } S$ 
40:          $p \leftarrow \text{Get } i\text{-th placeholder from } P$ 
41:         Add edge  $(\text{parent}(p), s)$  to  $G'$ 
42:       end for
43:     end if
44:   end if
45:   return  $G'$ 
46: end procedure

```

sequence template attacks on a larger real-world graph. As shown in Table 9, poisoning and evasion exhibit a substantial performance gap: poisoning is consistently more effective

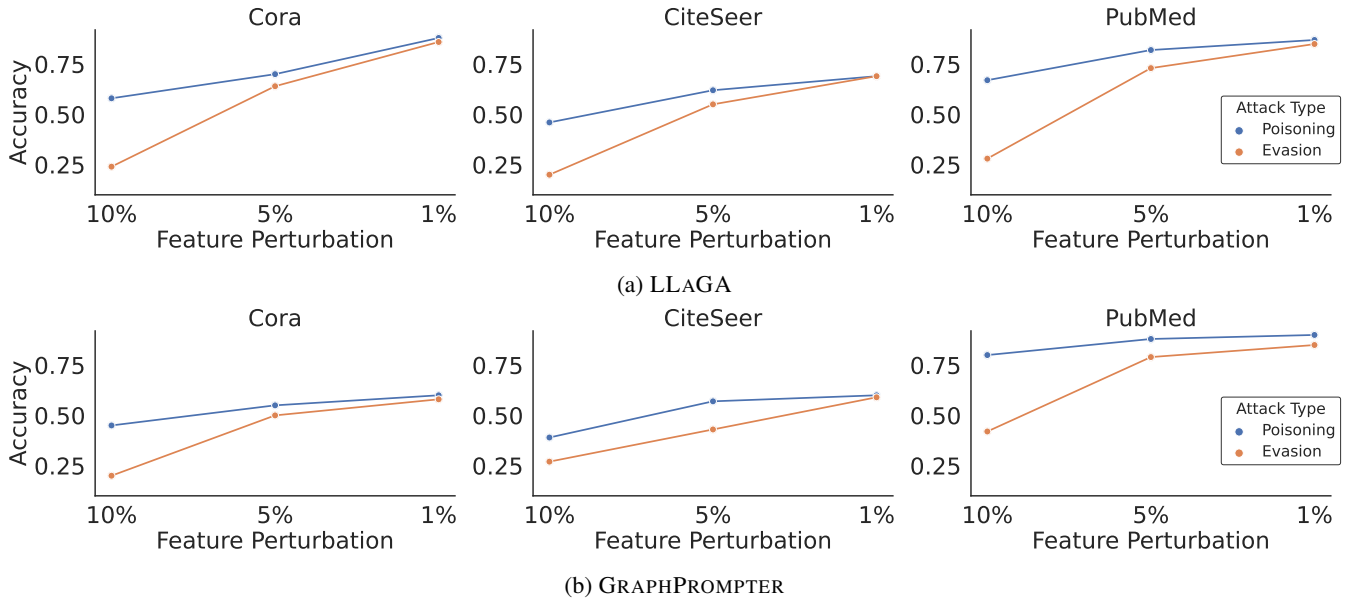


Figure 6: Accuracy of graph-aware LLMs as the feature perturbation percentage varies. Lower accuracy indicates a more effective attack.

Table 8: Effect of changing the base LLM on LLAGA accuracy under clean and attacked settings. The results indicate that base-model choice does not substantially change attack susceptibility.

Data	Base Model	Clean	Poisoning			Evasion		
			NETTACK	METAATTACK	MSI	NETTACK	METAATTACK	MSI
Cora	Vicuna-7B	0.89 ± 0.07	0.87 ± 0.04	0.79 ± 0.03	0.82 ± 0.04	0.55 ± 0.09	0.44 ± 0.06	0.30 ± 0.05
	Llama2-7B	0.90 ± 0.03	0.86 ± 0.02	0.77 ± 0.05	0.82 ± 0.02	0.56 ± 0.07	0.43 ± 0.08	0.31 ± 0.01
Citeseer	Vicuna-7B	0.71 ± 0.04	0.64 ± 0.05	0.63 ± 0.04	0.62 ± 0.02	0.59 ± 0.04	0.50 ± 0.03	0.24 ± 0.05
	Llama2-7B	0.71 ± 0.05	0.65 ± 0.03	0.60 ± 0.03	0.61 ± 0.04	0.58 ± 0.03	0.48 ± 0.05	0.26 ± 0.03
PubMed	Vicuna-7B	0.90 ± 0.04	0.89 ± 0.03	0.89 ± 0.08	0.81 ± 0.03	0.84 ± 0.05	0.73 ± 0.04	0.76 ± 0.03
	Llama2-7B	0.91 ± 0.06	0.88 ± 0.04	0.90 ± 0.02	0.80 ± 0.04	0.85 ± 0.03	0.75 ± 0.02	0.77 ± 0.02

than evasion across all three injection strategies, unlike the pattern observed on Cora, Citeseer, and PubMed. This suggests that ArXiv may have structural properties that increase poisoning vulnerability while preserving greater evasion robustness. Despite the low sparsity of ArXiv, inserting a single supernode can substantially affect model performance.

A.7 Feature Attack Examples and Defense Template

Feature Attack Examples The prompt examples in Appendix A.7 instantiate the adversarial attacks on real Cora node features. The "input" corresponds to the text received by the LLM after the attack is applied. Although a human reader perceives the original and perturbed text as visually identical or highly similar, the LLM receives either different Unicode code points or a different character ordering, which can change model behavior. Table 10 illustrates the Homoglyph attack, and Table 11 illustrates the Reordering attack.

Table 9: Accuracy under node sequence template injection attacks across datasets. *NI*, *SI*, and *MSI* denote non-adjacent injection, single-supernode injection, and multiple-supernode injection, respectively. Clean denotes model performance without attack.

Data	Attack	NI	SI	MSI
Cora	Clean	—	0.89 ± 0.07	—
	Poisoning	0.86 ± 0.03	0.84 ± 0.06	0.82 ± 0.04
	Evasion	0.42 ± 0.08	0.36 ± 0.05	0.30 ± 0.05
Citeseer	Clean	—	0.71 ± 0.04	—
	Poisoning	0.69 ± 0.05	0.66 ± 0.03	0.62 ± 0.02
	Evasion	0.36 ± 0.06	0.30 ± 0.04	0.24 ± 0.05
PubMed	Clean	—	0.90 ± 0.04	—
	Poisoning	0.88 ± 0.04	0.85 ± 0.05	0.81 ± 0.03
	Evasion	0.85 ± 0.07	0.82 ± 0.04	0.76 ± 0.03
ArXiv	Clean	—	0.73 ± 0.02	—
	Poisoning	0.50 ± 0.03	0.43 ± 0.04	0.42 ± 0.03
	Evasion	0.71 ± 0.06	0.69 ± 0.02	0.60 ± 0.03

Table 10: Illustration of the imperceptible Homoglyph attack. The LLM processes the perturbed input as distinct Unicode characters, while the rendered text remains nearly identical to a human reader.

Original Feature	What LLM Sees	What a Human Sees
Title: The megaprior heuristic for discovering protein sequence patterns Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.	Title: The mgpprir huristic fr discvring prtin squnc pptrns Abstract: Svrl cmputr lgrithms fr discvring pptrns in grups f prtin squncs r in us tht r bsd on fitting th prmtrs f sttisticl mdl to grup of rltd squncs.	Title: The megaprior heuristic for discovering protein sequence patterns. Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.

Table 11: Illustration of the imperceptible Reordering attack. The LLM processes a modified character order, while the rendered text remains nearly identical to a human reader.

Original Feature	What LLM Sees	What a Human Sees
Title: Applications of machine learning: a medical follow up study. Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-1R, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.	Title: Appliatcions of machine lreanign: a medical foall-up study Abstract: This ppaer dsceribes pmerlinary work that aims to aplpy smoe learning sttraegies to a meidcal flolow-up study. An inestivagiotn of the aplcpiation of three machine larening algorithms-1R, FOIL and InductH to identify risk frcatos that govern the clopsuspension cure rtae has been mdae.	Title: Applications of machine learning: a medical follow up study Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-1R, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.

Feature Corrector Defense Template We design a feature-corrector module to mitigate imperceptible attacks, specifically Homoglyph substitutions and character Reorderings. The module is implemented as a prompt-based LLM defense. Its prompt instructs the LLM to function as a linguistic validator that detects and corrects subtle perturbations that are difficult for humans to perceive.

The prompt template is shown below. It explicitly specifies the correction task and output format, which improves the reliability of the LLM correction procedure.

A.8 Detailed Discussion

The empirical findings identify several robustness and design considerations for graph-aware LLMs. As graph-structured data is increasingly integrated into LLM pipelines, evaluation must account for both performance gains and adversarial

failure modes. Our systematic study of LLAGA and GRAPH-PROMPTER shows that these architectures exhibit substantial weaknesses under poisoning and evasion attacks originally designed for GNNs. The main observations are discussed below.

Impact of Graph Encoding on Adversarial Vulnerability

A central observation is that the graph-structure integration mechanism can itself increase adversarial susceptibility. In LLAGA, the node sequence template creates an exploitable interface that permits substantial performance degradation. This attack surface, specific to LLAGA, illustrates a design tradeoff: node-level templates can encode graph structure effectively and improve graph-task performance, but they also introduce attack vectors absent from traditional GNNs. GRAPH-PROMPTER, which uses a GNN-based projector for graph encoding, exhibits comparatively stronger resilience,

Prompt for Feature Corrector

You are a linguistic validator. Your task is to detect and correct any subtle, imperceptible text perturbations such as homoglyph substitutions (e.g., replacing "o" with "0") and character reorderings. These changes are often used to attack machine learning models. Given a text string, carefully analyze it. If you identify any perturbations, provide the corrected, original-looking version of the text. If no perturbations are found, return the original text unchanged.

Example 1 (Homoglyph Substitution):

Input: Title: The mgprir heuristic fr discvring prtin squnc pttrens Abstract: Svrl cmputr lgrithms fr discvring pttrens in grups f prtin squncs r in us tht r bsd on fitting th prmtrs f sttisticl mdl to grup of rlted squncs.

Output: Title: The megaprior heuristic for discovering protein sequence patterns Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.

Example 2 (Character Reordering):

Input: Title: Appliatcions of machine lreanign: a medical foall-up study Abstract: This ppaer dscribes pmerlinary work that aims to aplpy smoe learning sttraegies to a meidcal flolow-up study. An inestivagiottn of the apclpiation of three machine larening algorithms-lR, FOIL and InductH to identify risk frcatos that govern the clopsuspension cure rtae has been mdae.

Output: Title: Applications of machine learning: a medical follow up study Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-lR, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.

Example 3 (Mixed Homoglyph & Reordering):

Input: Title: Submitted to NIPS96, Sctin: Applications. Prfrnc: Oral prsentatin Reinforcement Lrning for Dynmic Channl Allocation in Abstract: In ellulr tlphne systms, n imrtnt prblm is to dynmiclly alloct th cmmunictin rsurc (chnnls) so as to mximize srvc in stchstic cllr nvirnmt. This prblem is naturally frmultd s dynmic prgrmming prblm and we us a reinforcement lerning (RL) mthd to find dynmic chnnl llection plicies that ar bttr than prvius heuristic slutins.

Output: Title: Submitted to NIPS96, Section: Applications. Preference: Oral presentation Reinforcement Learning for Dynamic Channel Allocation in Abstract: In cellular telephone systems, an important problem is to dynamically allocate the communication resource (channels) so as to maximize service in a stochastic caller environment. This problem is naturally formulated as a dynamic programming problem and we use a reinforcement learning (RL) method to find dynamic channel allocation policies that are better than previous heuristic solutions.

Your Task: Text to correct: {perturbed_text_here}

Corrected Text:

Figure 7: Prompt template used by the LLM-based feature corrector. The template specifies the role, correction task, and expected output format.

suggesting that hybrid LLM-GNN architectures may provide robustness advantages.

Sensitivity of Graph-aware LLMs to Feature Perturbations Beyond graph-encoding vulnerabilities, the results show that feature perturbation attacks are effective against both graph-aware LLMs. Although LLAGA and GRAPH PROMPTER encode structure differently, both are sensitive to imperceptible perturbations of textual node features. This reflects a broader LLM robustness issue: models that process text effectively may still be vulnerable to character-level or rendering-level adversarial manipulations. In security-sensitive applications, the ability to degrade predictions through visually subtle node-attribute changes threatens model integrity. Text sanitization methods such as those discussed by (Boucher et al. 2022) provide one defense direction, but imperceptible attacks such as homoglyph manipulations remain difficult to remove reliably. Our proposed LLM corrector mitigates this vulnerability by detecting and repairing suspicious or malformed inputs before downstream graph-aware processing.

Data-dependent Variability of Poisoning and Evasion Attacks on Graph-aware LLMs The effectiveness of poisoning and evasion attacks is dataset dependent. Although attacks are strong on Cora, Citeseer, and PubMed, the ArXiv results show a different interaction between attack strategy and graph structure. This variability indicates that robustness evaluation must account for dataset properties rather than relying only on model architecture. As graph-aware LLMs are applied across domains, vulnerability assessments should therefore consider both algorithmic design and the structural/statistical properties of the target data.

Implications for Graph-aware LLM Development The observed vulnerabilities have direct implications for the design and deployment of graph-aware LLMs. As these models are adopted for tasks previously handled by GNNs, their encoding mechanisms must be evaluated not only for accuracy but also for adversarial exposure. Future systems require more robust graph encoders and defenses against feature-level manipulation. Understanding how GNN attacks transfer to graph-aware LLMs also defines a research direction for protecting these models against stronger and more adaptive adversaries.

Difficulty in Applying Existing GNN defenses Against Adversarial Attacks on Graph-aware LLMs **Defenses on adversarial attacks based on feature perturbation.** Adversarial attacks on GNNs have primarily targeted structural perturbations because feature perturbations often do not change the predicted class of the target node (Dai et al. 2018; Wu et al. 2019; Zügner and Günnemann 2019). As a result, there are *no known defenses against feature-based attacks on GNNs* in the same sense, because such attacks have limited effectiveness in the traditional GNN setting. In contrast, the main paper shows that feature perturbations are effective in the graph-aware LLM paradigm. This creates a distinct defense problem: feature-based attacks that were previously ineffective against GNNs can manipulate graph-aware LLM behavior. Defenses must therefore

address both structural and feature-level perturbations. Text sanitization methods, such as optical character recognition (OCR) preprocessing before LLM encoding, are possible mitigations, but imperceptible attacks such as homoglyph substitutions remain difficult because OCR is imperfect and misinterprets homoglyphs at a higher rate than unperturbed text (Boucher et al. 2022). This limitation motivates our LLM-based self-correction mechanism for feature perturbations.

Defenses on adversarial attacks based on structural perturbation. Because structure-based adversarial attacks transfer from GNNs to graph-aware LLMs, especially in evasion settings, one might expect GNN defenses to transfer as well. The empirical results do not support this assumption. Unlike traditional GNNs, *graph-aware LLMs are also susceptible to feature perturbations, rendering these defenses ineffective.*

One GNN defense mechanism makes the adjacency matrix trainable so that edge weights can adapt during training. This approach downweights edges between dissimilar nodes and upweights edges between nodes with higher feature similarity. Several **preprocessing-based** defenses instantiate this principle, including removal of edges with low feature similarity (Wu et al. 2019) and randomized edge dropping (Dai et al. 2018). These defenses, however, depend on clean node features for reliable similarity estimation. When node features are also perturbed, the similarity scores become unreliable, and the defense may fail or even introduce incorrect edges. Figure 8 demonstrates this effect for a random Cora node. The clean feature-similarity graph preserves some original edges while adding some incorrect connections. By contrast, reconstruction from perturbed features preserves no original connections, producing an entirely incorrect local structure.

Another defense line reduces dependence on graph structure. For example, (Shanthamallu, Thiagarajan, and Spanias 2021) proposed a surrogate predictor that depends only on node features and uses uncertainty matching to extract graph information from the GNN. Wu et al. (2021) introduced a co-training framework that trains submodels separately on feature and structural views, with knowledge distillation supported by confident unlabeled examples. These methods also assume access to unperturbed node features. That assumption fails in our setting, where both structure and features may be adversarially modified. Consequently, structural defenses developed for GNNs are not directly applicable to graph-aware LLMs and must be redesigned for this paradigm.

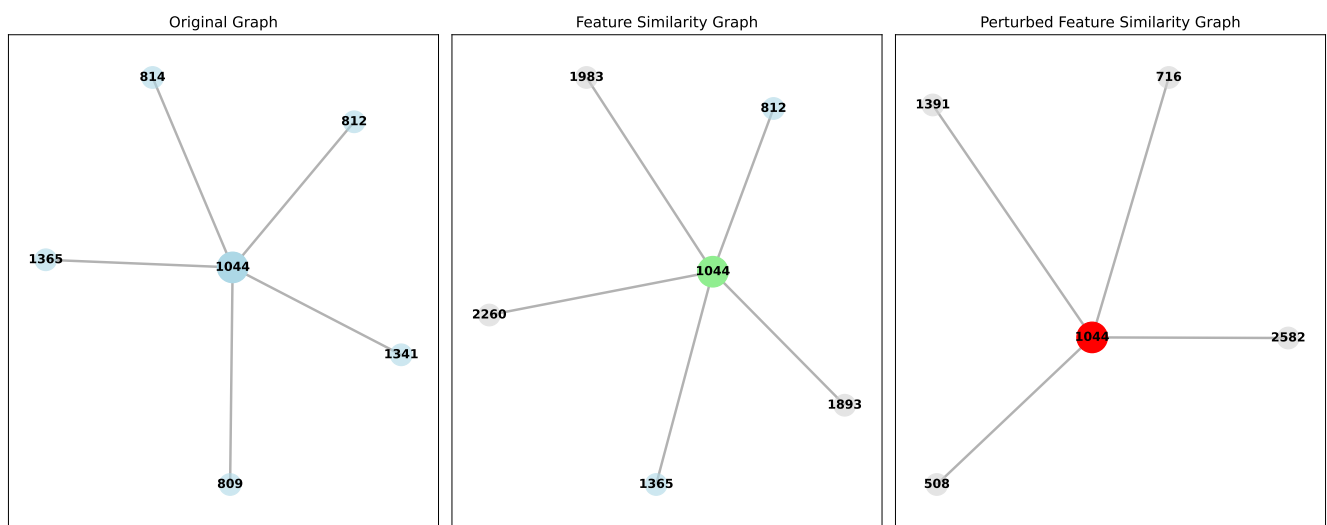


Figure 8: Edges reconstructed for random node 1044. **Original (left)** shows ground-truth edges from the original graph. **Feature Similarity (middle)** shows edges reconstructed from clean node features, with blue nodes denoting preserved original connections. **Perturbed Feature Similarity (right)** shows edges reconstructed from perturbed node features.